

The Agent Stack - Part 4: Runtimes, Workflows, and Durable Execution

Why useful agents need more than a loop



VINOTH GOVINDARAJAN

APR 20, 2026



9



1

Share

An agent starts a dependency upgrade.

It edits files, runs tests, pushes a branch, waits for CI, asks a maintainer for approval, and opens a PR.

Then the worker restarts while CI is still running.

The interesting question is not whether the model can suggest the next command. The interesting question is what the system knows when it comes back.

Did the branch already get pushed?

Which tests belong to this run?

Is the approval still valid after the diff changed?

Which step can safely run again?

That is the runtime layer.

Not the model. Not memory. Not the chat UI.

The runtime advances the run. A workflow gives that run a recoverable shape. Durable execution records enough progress for the work to survive time, failure, waiting, and retry.

■ A loop can answer a turn. A durable workflow can survive time.

The runtime owns progress

In [Part 3](#), the control plane owned the source of truth for the run.

It decided what run exists, which session it belongs to, what policy applies, and where state should live.

The runtime has a different job.

It moves the run forward.

That sounds small until the run starts doing real work.

A runtime assembles context, calls the model, interprets model output, invokes tools, handles handoffs, pauses when approval is needed, resumes from saved state, and emits evidence about what happened.

One way to see the shape is the OpenAI Agents SDK. OpenAI's docs frame it as a fit when your application owns orchestration, tool execution, approvals, state, and storage. They also point from the high-level Agents guide into running agents, handoffs, human review, results/state, and tracing. That is runtime territory.

That does not mean the runtime owns everything.

It does not decide who the user is.

It does not decide what a tenant is allowed to do.

It does not turn memory into truth.

It does not make a browser, shell, database, or payment API safe by default.

But it does own the execution path.

Given the current state of the run, the runtime decides the next operational move:

Call the model.

Call a tool.

Hand off.

Pause.

Resume.

Return.

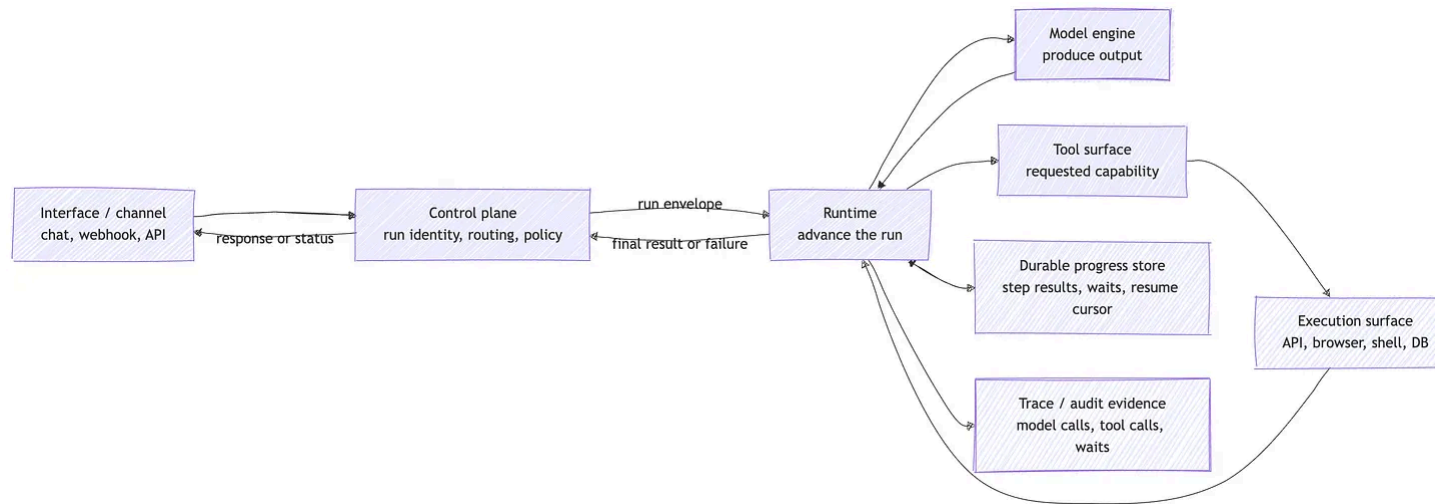
Fail.

Retry.

The useful distinction is this: the runtime owns progress, not ultimate authority.

The control plane owns the source of truth around the run. The model engine produces output. The runtime turns that output into ordered progress.

If you remember one picture from this post, make it this one: the runtime advances the run, but it does not own every layer around it.



The runtime is not just the code that runs the agent. It is the layer that turns model output into ordered progress.

The loop is the hot path, not the system

The demo version of an agent often looks like a loop.

Take input.

Call the model.

Maybe call a tool.

Append the tool result.

Call the model again.

Return an answer.

That loop is real. OpenAI's Agents SDK docs describe state strategies for multi-turn runs, and the SDK's results and tracing surfaces make it clear that runs can involve handoffs, interruptions, and richer run state than a single prompt-response exchange.

For a single turn, this is often enough.

The user asks a question. The agent retrieves a document. The model summarizes it. The runtime returns the answer.

No one has to wait for a webhook.

No external approval arrives later.

No worker needs to recover state after a deploy.

No side effect has to be protected from duplicate execution.

Now change the task.

The agent starts a repo migration. It opens a branch, modifies files, runs tests, waits for CI, posts a status update, and asks a maintainer before opening the PR.

This is still one task from the user's point of view.

It is not one request from the system's point of view.

There are pauses. There are external events. There are tool outputs. There are partial side effects. There may be retries. There may be a restart.

The loop is necessary. It is just not the whole contract.

The larger problem is progress across time.

What has already happened?

What is still pending?

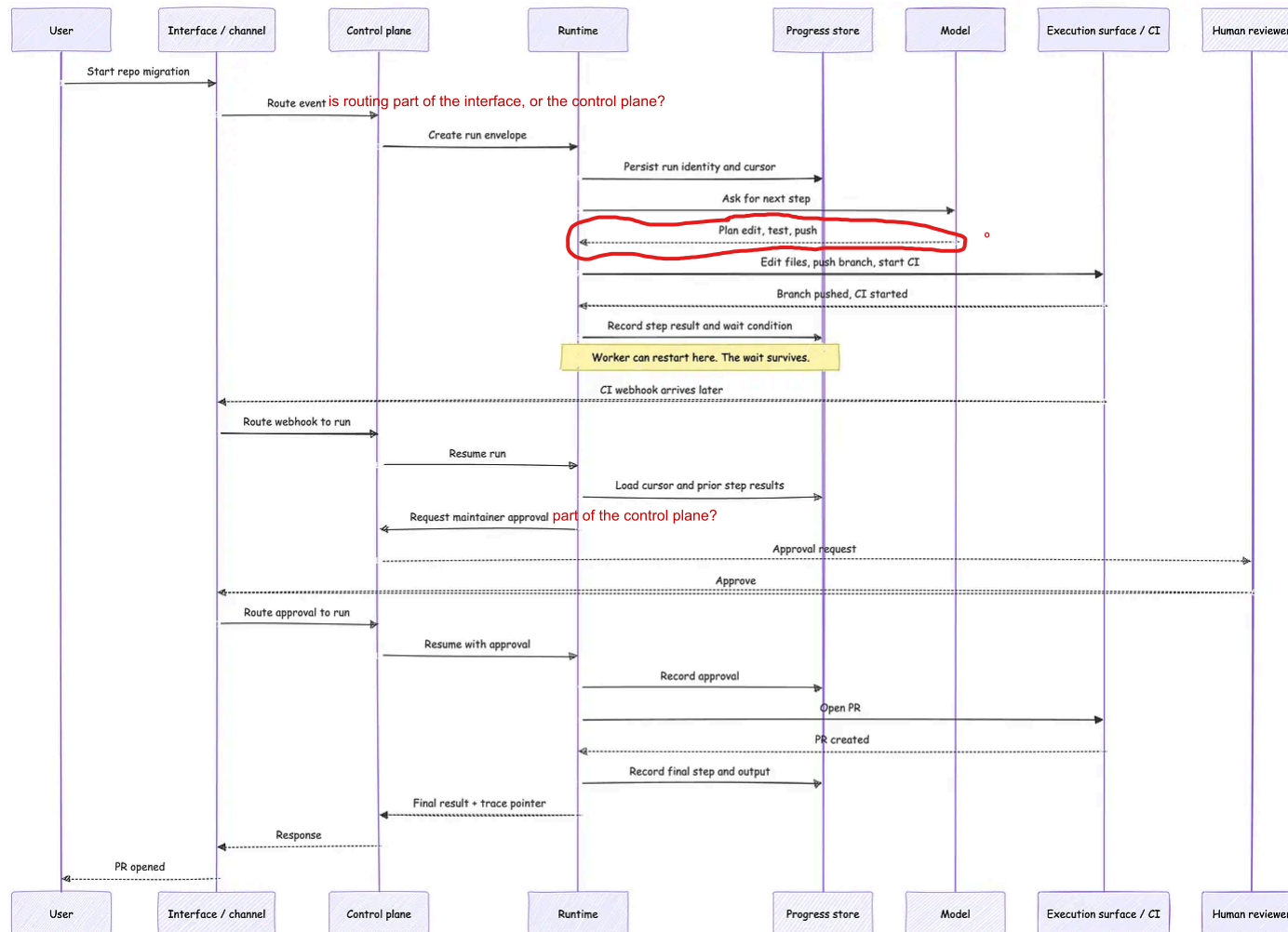
What can run again?

What must not run again?

What does the operator see when the run is stuck?

Once those questions show up, "just keep looping" is not enough.

From the user's point of view, this is one task. From the system's point of view, it is a run with state, waits, side effects, and recovery points.



The important detail is not the CI system or the PR. The important detail is that every external wake-up re-enters through the control plane and resumes a specific run.

Workflows appear when progress needs a shape

A workflow is often explained as a sequence of steps.

That is not wrong. It is just incomplete.

In agent systems, the more useful definition is this:

A workflow is the recoverable shape of a run.

It tells the runtime what states are possible, where work is allowed to happen, where side effects are isolated, where waits occur, and how execution continues after interruption.

Different systems package this differently. LangGraph makes the checkpoint and interrupt model explicit: its persistence layer saves graph state as checkpoints organized into threads, and its interrupt model pauses execution, saves state, waits indefinitely, and resumes later. Restate and DBOS make the same pressure visible through journals and stored step results; OpenAI's own Agents SDK docs point to DBOS integrations for long-running agents, human-in-the-loop flows, and preserved progress across failures and restarts. The product shapes vary. The boundary does not.

The workflow has to answer a few practical questions:

What is the run identity?

Where is progress recorded?

Which steps have completed?

Where can the run wait?

What external events can wake it up?

What can retry?

What requires approval?

What evidence is emitted when something fails?

That is why a workflow is not just a list of steps.

A list tells you what you hoped would happen.

A workflow tells the runtime how to continue when reality interrupts.

Durable execution records before ambiguity

Durable execution can sound bigger than it is.

The plain version is this:

The system records enough progress that a run can stop, wait, fail, restart, and continue without pretending the whole thing is new.

The key word is progress.

Not transcript.

Not memory.

Not “the model remembers.”

Progress.

Temporal’s Event History is a clean example. Temporal describes Event History as a durable log of what happened during a Workflow Execution, and its SDK docs explain that replay uses that persisted history to rebuild workflow state after failures and resume execution. LangGraph makes a similar point from a graph-oriented direction: durable execution preserves completed work so a workflow can resume without reprocessing previous steps, and it requires deterministic workflow logic with side effects wrapped behind task boundaries.

Different implementation, same architectural pressure.

Record progress before the world becomes ambiguous.

That is the invariant.

The ambiguity usually comes from side effects.

A model call can be repeated and give a different answer. That may be annoying.

A branch push, payment call, email send, ticket update, or database write can be repeated and change the outside world twice. That is worse.

This is where people collapse four ideas that should stay separate.

Retry means re-attempting failed work.

Replay means rebuilding workflow state from recorded history or checkpoints.

Resume means continuing after a wait, approval, crash, reconnect, or restart.

Idempotency means a repeated operation does not create an unintended additional effect.

Those concepts are related.

They are not interchangeable.

A retry without idempotency can duplicate a side effect.

A replay without step boundaries can repeat work that should have been recorded.

A resume without a persisted cursor can invent context.

An idempotency key without a workflow still does not tell you which step of the larger run is waiting.

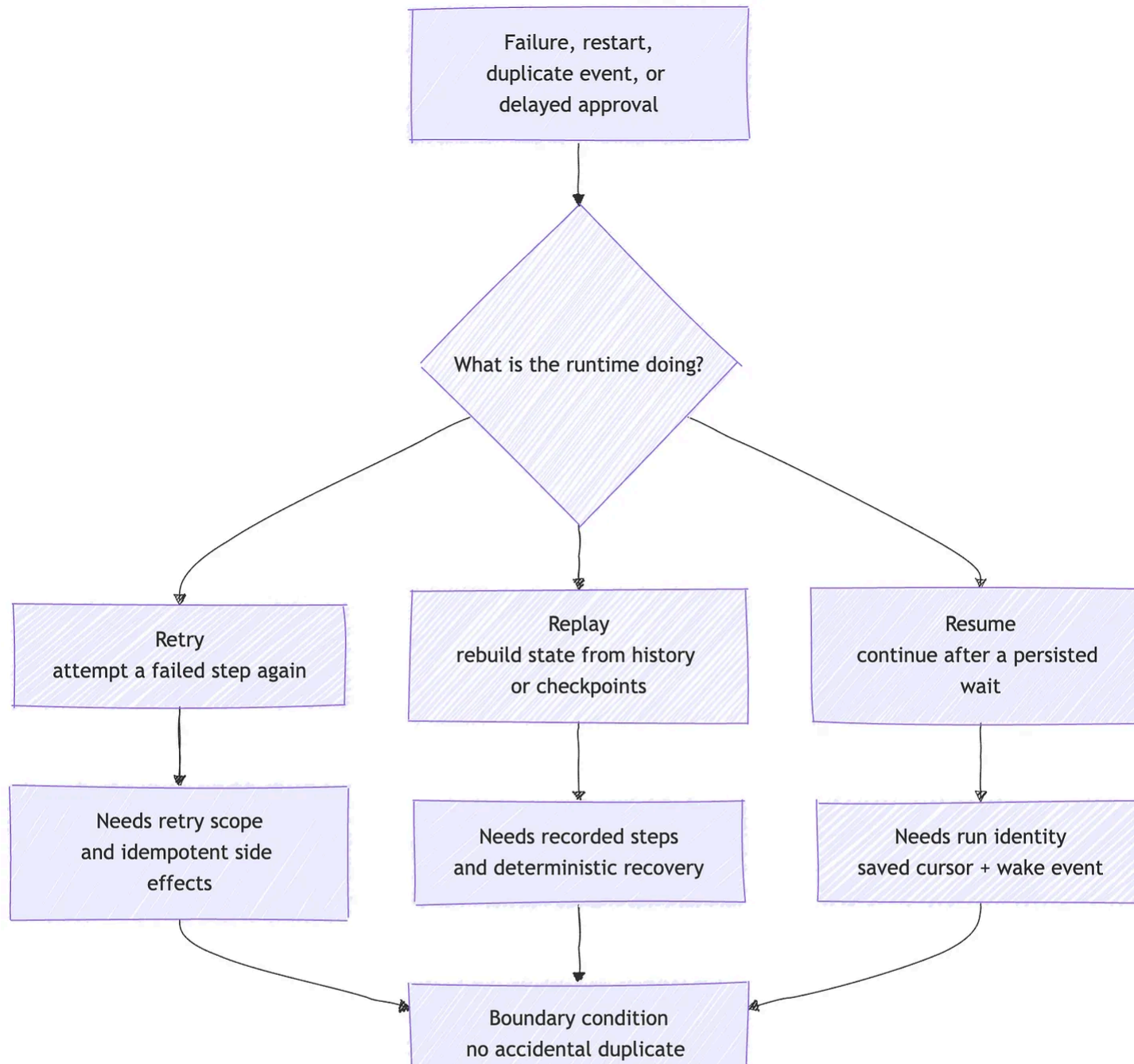
Temporal's retry docs make one part of that boundary concrete: retry policy tells Temporal how and when to try again after failure, while workflow code is expected to remain deterministic and activities are where failure-prone side effects usually live.

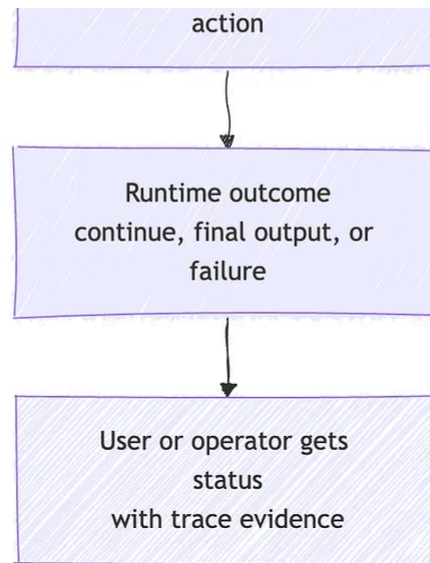
Queues are another good place to see the distinction.

SQS standard queues provide at-least-once delivery, which means more than one copy of a message might be delivered and messages may occasionally arrive out of order. AWS explicitly tells developers to design for duplicate processing and use idempotent operations. That is a queue guarantee. It is not a workflow guarantee. A queue can deliver work. A workflow can tell you where the larger run is. You often need both.

■ Durable execution is recorded progress before the world becomes ambiguous.

These recovery terms often get used together, but they solve different problems.





Waiting is not sleeping

Long-running agent work spends a lot of time not running.

Waiting for CI.

Waiting for a human.

Waiting for a webhook.

Waiting for a timer.

Waiting for another service to finish.

A weak runtime treats this as a blocked process or a loose background task.

A stronger runtime treats waiting as state.

OpenAI's human-in-the-loop docs make this explicit: when a tool call requires approval, the SDK pauses the run, returns interruptions, and lets you resume later from the same run state. LangGraph's interrupts express the same shape: execution pauses, state is saved through the persistence layer, and the graph resumes later with external input.

This matters because approval is not just a button.

Approval is a decision attached to a specific pending action in a specific run.

The runtime needs to know:

What action is pending?

What state produced it?

Who can approve it?

What changed while it waited?

What resumes after the answer arrives?

What happens if the answer never arrives?

The same shape applies to external events.

A CI webhook is not just “new input.” It is evidence that may unblock a specific run, branch, commit, and step.

A timer is not just “sleep.” It is a persisted wait condition.

A background model task is not automatically durable workflow execution. OpenAI’s background mode is about running long-running model tasks asynchronously and checking status later. That helps with timeouts and connectivity, but it is a different boundary from a multi-step workflow that records side effects, approvals, retries, and resume points.

Waiting is cheap only if it is represented correctly.

Otherwise, it becomes a pile of dangling state.

Keep the adjacent layers separate

A good runtime can make an agent system feel much more reliable.

That does not mean the runtime should absorb the whole stack.

This is where a lot of category mistakes start.

Checkpointed state is not the same thing as memory.

A checkpoint answers: where was this run?

Memory answers: what should the system know later?

LangGraph's docs make this distinction visible. Its persistence layer saves execution state as checkpoints, while its memory model distinguishes short-term working memory inside an ongoing execution from longer-lived memory across sessions.

Tool exposure is not the same thing as execution authority.

A runtime may expose a tool schema to the model. The actual action may happen in a browser, shell, database, API, code sandbox, or remote worker. That execution surface has its own risks, credentials, isolation model, and audit requirements.

Handoff is not the same thing as authorization.

A handoff can move execution to another agent. It should not silently move permission. OpenAI's handoff docs describe delegation between agents, represented as tools to the model. That is an execution-routing surface, not a complete trust model.

Tracing is not the same thing as evaluation.

Tracing gives you evidence. OpenAI's tracing docs describe traces that include model generations, tool calls, handoffs, guardrails, and custom events. That helps you debug and monitor a run. It does not automatically tell you whether the behavior was correct.

This is why the stack needs separate layers.

The runtime advances work.

Memory shapes what context can be reused later.

Tools define what capabilities the model can request.

Execution surfaces are where actions happen.

Identity and policy decide what is allowed.

Observability records what happened.

Evaluation decides whether the behavior was good enough to keep shipping.

Collapse those together and the system becomes hard to reason about.

Keep them separate and the design gets easier to inspect.

Failure modes

Retrying the whole agent after a partial side effect

The agent pushes a branch, opens a ticket, sends an email, or calls a payment API.

Then a later step fails.

If the system retries the whole run from the beginning, it may repeat the side effect.

The fix is not a better prompt.

The fix is a step boundary, recorded progress, and idempotency around the external action.

Treating a queue as a workflow

A queue can buffer work, smooth spikes, and decouple producers from consumers.

It does not automatically know that step four of a seven-step run already completed.

SQS standard queues provide at-least-once delivery, which means your application must tolerate duplicate message processing. That is a delivery property, not a full workflow model.

Use queues.

Just do not confuse delivery with progress.

Treating background execution as durability

Running a task in the background can avoid client timeouts.

It does not automatically give you replay, step-level recovery, approval persistence, or side-effect dedupe.

A background task can still lose its place.

Durable execution means the system knows what completed, what is waiting, and what can safely happen next.

Treating approval as a modal

A button is not an approval system.

The approval needs to be tied to the pending action, run identity, current state, approver identity, and resume path.

Otherwise the system can approve the wrong thing, approve stale state, or resume after the world has changed.

Replaying non-deterministic work

Replay should rebuild workflow state.

It should not reroll random choices, repeat external API calls, rewrite files, or resend messages unless those operations are wrapped behind durable step boundaries.

Temporal and LangGraph both push toward the same discipline here: workflow logic must be replayable, and side effects belong behind explicit boundaries.

Letting handoff move authority

An agent can hand off work to another agent.

That does not mean the new agent should inherit every permission, credential, memory, or tool.

Handoff is runtime routing.

Authority belongs to the policy and identity layers.

Hiding the evidence

A run fails.

No one can tell which model call made the decision, which tool ran, which approval was pending, which event resumed the workflow, or which side effect already

happened.

That is the 3:00 AM version of “the agent feels haunted.”

The model is not the mystery.

The missing evidence is.

Builder checklist

When you design the runtime layer, ask these questions.

1. **What is the run identity?**

Every meaningful execution needs a stable ID that ties together input, session state, tool calls, waits, approvals, retries, output, and trace evidence.

2. **Where is progress recorded?**

Do not rely on process memory for work that can wait, retry, or resume.

3. **What are the step boundaries?**

Put side effects, expensive work, external calls, and non-deterministic operations behind boundaries the runtime can record.

4. **What can safely run twice?**

Design idempotency before production side effects. Email sends, ticket creation,

branch pushes, database writes, and payment calls all need explicit repeat behavior.

5. What does waiting mean?

Classify waits: human approval, timer, webhook, external job, queue event, long-running model task, or tool execution.

6. What wakes the run back up?

A resume event should identify the run, the pending wait, the current state, and the input that satisfies the wait.

7. Where does approval attach?

Approval should attach to a specific pending action, not to a vague conversation or agent response.

8. What evidence will an operator have?

Emit traces for model calls, tool calls, handoffs, waits, approvals, retries, resumes, failures, and final output.

Recap

A loop is the starting point.

It is not the system.

Useful agents often do work that crosses time. They wait for humans, tools, web hooks, timers, background jobs, and external systems. They fail halfway through. They retry. They resume. They create side effects that should not repeat by accident.

The runtime layer exists to advance that work sanely.

A workflow gives the run a recoverable shape.

Durable execution records progress before ambiguity enters the system.

Once you see that boundary, agent execution stops looking like a magic conversation.

It starts looking like a stateful system with model calls inside it.

That is a much better place to build from.

What comes next

Next is Part 5: **Context, Retrieval, and Memory**.

That layer answers a different question.

Runtime asks: where is this run in its execution?

Context asks: what should the model see right now?

Memory asks: what should the system preserve and re-inject later?

Those questions are adjacent.

They should not be collapsed.

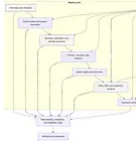
If this kind of systems-level agent architecture is useful, subscribe to follow the rest of the series.

References

- [OpenAI Agents SDK](#): runs, handoffs, approvals, and tracing
- [OpenAI Agents SDK, human-in-the-loop](#): pause, approval, and resume
- [LangGraph Docs](#): durable execution, checkpoints, and resumability
- [Temporal Docs](#): workflow execution, replay, and recovery
- [Amazon SQS Docs](#): at-least-once delivery and idempotent consumers
- [Restate Docs](#): journals, durable steps, and replay

The Agent Stack v1

New here? Start with [Part 1](#) for the full stack map. Subscribe if you want the rest of the series as it publishes.



The Agent Stack - Part 1: A Systems Map of Modern Agent Infrastructure

VINOTH GOVINDARAJAN • 30 MAR

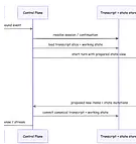
[Read full story](#) →



The Agent Stack - Part 2: Foundation Infrastructure, Models, and Inference

VINOTH GOVINDARAJAN • 6 ABR

[Read full story](#) →



The Agent Stack - Part 3: Control Planes, Sessions, and State Ownership

VINOTH GOVINDARAJAN • 13 ABR

[Read full story](#) →



9 Likes · 1 Restack

Discussion about this post

Comments

Restacks



Write a comment...